



Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Estado de México

Implementación con framework de Random Forest para clasificación binaria y
multiclase.

Inteligencia Artificial avanzada para la ciencia de datos

Roberd Otoniel Meza Sainz

A01801490

Grupo 601

Introducción y objetivo

El objetivo de esta entrega es implementar el mismo algoritmo de clasificación de la entrega anterior, Random Forest, pero ahora usando un framework de machine learning en vez de programarlo desde cero. Se usó scikit-learn, en específico RandomForestClassifier de sklearn.ensemble, para demostrar el manejo y la configuración correcta de sus hiperparámetros.

El RandomForestClassifier se configuró con hiperparámetros equivalentes a los de la implementación from scratch, para que ambas versiones queden bajo condiciones comparables:

- `min_samples_split = 2`
- `max_depth = 5`
- `max_features = 'sqrt'` (equivalente a `sqrt(n_features)` usado a mano)
- `criterion = 'gini'`
- `bootstrap = True`

Al igual que en la entrega anterior, se probó la implementación en los mismos cuatro datasets y en el mismo orden creciente de dificultad/tamaño:

- Iris (binario: versicolor vs virginica).
- Breast Cancer Wisconsin (Diagnóstico): problema binario real de tamaño mediano (569 muestras aprox, 30 features).
- Wine: problema multiclase de 3 clases.
- MAGIC Gamma Telescope: dataset binario “grande” (19,020 muestras aproximadamente).

Algoritmo

El código conserva la misma estructura general que la versión from scratch (carga de datos, partición train/test, cross validation, entrenamiento y evaluación), pero reemplazando las clases DecisionTree y RandomForest hechas a mano por las herramientas de scikit-learn.

Split_dataset(X, y, test_size, random_seed)

Esta función se copió tal cual de la entrega anterior en vez de usar `train_test_split` de sklearn. La razón es que, aunque se use la misma semilla, `train_test_split` separa el test tomando el inicio del arreglo permutado, mientras que esta función toma el final, lo que produce un test set distinto aunque el tamaño coincida. Al reutilizar la misma función, el train/test de esta entrega queda idéntico al de la entrega anterior.

```
def split_dataset(X, y, test_size=.2, random_seed=42):
    np.random.seed(random_seed)
    idx = np.random.permutation(len(X))
    corte = int(len(X) * (1 - test_size))
    train_idx, test_idx = idx[:corte], idx[corte:]

    return X[train_idx], X[test_idx], y[train_idx], y[test_idx]
```

Cross validation

En vez de la función `k_fold` hecha a mano, aquí se usa `KFold` de `sklearn.model_selection` con `n_splits=5`, `shuffle=True` y `random_state=42`. En cada fold se crea y entrena un `RandomForestClassifier` nuevo con los hiperparámetros ya mencionados, se predice sobre el fold de validación y se calcula el `accuracy` con `accuracy_score` de `sklearn.metrics`.

```
bosque_cv = RandomForestClassifier(
    n_estimators=numero_arboles,
    min_samples_split=2,
    max_depth=5,
    max_features='sqrt',
    criterion='gini',
    bootstrap=True,
    random_state=42
)
```

Evaluacion_multiclase(y_real, y_pred, titulo)

Función equivalente a la de la entrega anterior, con un parámetro `titulo` agregado para poder reutilizarla y distinguir entre distintos reportes. Calcula el `accuracy` global (`np.sum(y_real == y_pred) / total`) y construye la matriz de confusión con `pd.crosstab`.

```
def evaluacion_multiclase(y_real, y_pred, titulo="Resultados"):
    correctos = np.sum(y_real == y_pred)
    total = len(y_real)
    accuracy = correctos / total

    print(f"\n-- {titulo} --")
    print(f"Accuracy : {accuracy:.4f}")
    print(f"Acertados: {correctos} de {total}")
```

```

print("\n--Matriz de Confusion--")
matriz = pd.crosstab(pd.Series(y_real, name='Real'), pd.Series(y_pred,
name='Prediccion'))
print(matriz)
print("-----\n")

return accuracy

```

Entrenamiento y evaluación final

Se entrena un único RandomForestClassifier con los mismos hiperparámetros sobre todo el train set, se predice sobre el test set, y además del accuracy y la matriz de confusión se imprime un classification_report de sklearn.metrics, que da precision, recall y f1-score por clase de forma automática.

```

print(classification_report(y_test, predicciones))

```

Metodología

- Carga del dataset a X y y (misma función que la entrega anterior).
- Partición train/test: misma función split_dataset, semilla fija, 30% del dataset para test.
- Cross validation (KFold de sklearn, k=5, shuffle=True, random_state=42) sobre el conjunto de entrenamiento, entrenando un RandomForestClassifier nuevo en cada fold.
- Entrenamiento final del RandomForestClassifier sobre el conjunto de entrenamiento completo.
- Evaluación sobre el conjunto de prueba: accuracy, matriz de confusión y classification report.

Resultados

Dataset Iris binario: número de árboles = 15, max_depth = 5, min_samples_split = 2, k_fold = 5

Cross validation:

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Promedio
0.9286	0.9286	0.8571	0.9286	0.9286	0.9143

Evaluación en test (30 muestras): accuracy = 0.9667 (29 de 30 aciertos).

Matriz de confusión:

Real \ Predicción	0	1
0	10	1
1	0	19

Métricas:

Clase	Precision	Recall	F1-score	Support
0	1.00	0.91	0.95	11
1	0.95	1.00	0.97	19
accuracy			0.97	30
macro avg	0.97	0.95	0.96	30
weighted avg	0.97	0.97	0.97	30

Dataset Breast Cancer Wisconsin: número de árboles = 15, max_depth = 5, min_samples_split = 2, k_fold = 5

Cross validation:

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Promedio
0.9375	0.9125	0.9625	0.9494	0.9241	0.9372

Evaluación en test (171 muestras): accuracy = 0.9591 (164 de 171 aciertos).

Matriz de confusión:

Real \ Predicción	0	1
0	96	3
1	4	68

Métricas:

Clase	Precision	Recall	F1-score	Support
0	0.96	0.97	0.96	99
1	0.96	0.94	0.95	72
accuracy			0.96	171
macro avg	0.96	0.96	0.96	171
weighted avg	0.96	0.96	0.96	171

Dataset Wine: número de árboles = 101, max_depth = 5, min_samples_split = 2, k_fold = 5

Cross validation:

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Promedio
1.0000	1.0000	1.0000	1.0000	0.9583	0.9917

Evaluación en test (171 muestras): accuracy = 0.9591 (164 de 171 aciertos).

Matriz de confusión:

Real \ Predicción	1	2	3
1	15	0	0
2	0	21	1
3	0	0	17

Métricas:

Clase	Precision	Recall	F1-score	Support
1	1.00	1.00	1.00	15
2	1.00	0.95	0.98	22
3	0.94	1.00	0.97	17
accuracy			0.98	54
macro avg	0.98	0.98	0.98	54
weighted avg	0.98	0.98	0.98	54

Dataset MAGIC Gamma Telescope: número de árboles = 15, max_depth = 5, min_samples_split = 2, k_fold = 5

Cross validation:

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Promedio
0.8351	0.8231	0.8479	0.8374	0.8249	0.8337

Evaluación en test (5,706 muestras): accuracy = 0.8309 (4,741 de 5,706 aciertos).

Matriz de confusión:

Real \ Predicción	0	1
0	1184	840
1	125	3557

Métricas:

Clase	Precision	Recall	F1-score	Support
0	0.90	0.58	0.71	2024
1	0.81	0.97	0.88	3682
accuracy			0.83	5706
macro avg	0.86	0.78	0.80	5706
weighted avg	0.84	0.83	0.82	5706

Análisis y conclusiones

Tabla Comparativa

Dataset	# Clases	Train / Test	CV accuracy (5-fold)	Accuracy bosque (test)
Iris	2	70 / 30	0.9143	0.9667
Wisconsin	2	398 / 171	0.9372	0.9591
Wine	3	124 / 54	0.9917	0.9815
MAGIC	2	13,314 / 5,706	0.8337	0.8309

Wine fue el dataset con mejor desempeño con el framework: el cross validation dio prácticamente perfecto (99.17%, con un solo fold en 95.8%) y el test se mantuvo muy cerca (98.15%), con 101 árboles el bosque casi no comete errores entre las tres clases.

Wisconsin y MAGIC salieron consistentes entre cross validation y test (93.72% vs 95.91%, y 83.37% vs 83.09% respectivamente), lo que da confianza de que el modelo generaliza de forma pareja en ambos casos, aunque en MAGIC el nivel de accuracy general se queda claramente por debajo del resto.

Iris fue el más inconsistente: el cross validation dio el promedio más bajo entre los datasets “fáciles” (91.43%, con un fold cayendo hasta 85.71%), pero el test terminó siendo el más alto de los cuatro (96.67%). Con solo 70 muestras de train divididas en 5 folds (aprox 14 por fold), un solo error de clasificación mueve bastante el accuracy de ese fold, lo que explica el salto.

En MAGIC, aunque el accuracy general (83%) es el más bajo de los cuatro, el classification report muestra que el bosque tiene mucho más recall en la clase 1 (0.97) que en la clase 0 (0.58), es decir que atrapa casi todos los positivos pero también genera bastantes falsas alarmas sobre la clase negativa. Esto probablemente tiene más que ver con los hiperparámetros (max_depth=5, 15 árboles) que se usaron por tiempo de cómputo en un dataset de 19,020 muestras, que con una limitación del framework en sí.